Grace Jin

San Jose, CA | (408) 750-7200 | gsj33@cornell.edu | gracejin.dev | github.com/gracejinsotrue | linkedin.com/in/grace-jin-cornell

EDUCATION

Cornell University, College of Engineering — Expected December 2027
Master of Engineering, Computer Science

Cornell University, College of Engineering — Expected May 2027
B.S. Computer Science, Minor in Artificial Intelligence

- **Relevant Coursework:** Systems Programming, Compiler Design, Physically Based Rendering, Computer Graphics, Machine Learning, Data Structures and Algorithms, Digital Logic and Computer Organization
- **Leadership:** Cornell XR (Extended Reality) Project Team Co-Founder, Space Systems Design Studio Software Lead

EXPERIENCE

LinkedIn | Software Engineering Intern, Mountain View, CA — May 2026 – Present

- Systems Infrastructure team; build an autonomous LLM-agent autofixer that rewrites blocking database and filesystem I/O into async equivalents across 50+ production Python services, cutting per-request blocked thread time by 95%
- Own design and implementation of the autofixer's core agent, orchestrating deterministic build and validation tooling with deepagents into a LangGraph pipeline that shipped 100% traceable production pull requests at a 85% merge rate
- Build an anti-gaming guard stack (AST assertion diffing, fault-injection tests, post-build rescans) and bounded self-repair loop that treat each green build as adversarial, blocking gamed or broken code from passing as production-ready

Cepton Technologies | Software Engineering Intern, San Jose, CA — May 2025 – Aug. 2025

- Graphics Perception Team; built a real-time WGPU-accelerated 3D visualization pipeline on Linux systems to simulate LiDAR data rendered over synthetic autonomous vehicle driving geometries at 60+ FPS
- Implemented WGSL compute shaders to perform parallel raycasting, generating 3600+ labeled frames of ML model training data per simulation run
- Built a 3D environment reconstruction system in Rust to generate simulator depth maps integrated within GPU pipeline
- Replaced legacy Rust transformations with library-backed matrix kernels (SVD, affine), reducing compute time by 40%
- Authored engineering blog post documenting this project: developer.cepton.com/blog/AR_simulator

People and Robots Teaching and Learning (PoRTaL) | Software Developer, Ithaca, NY — Aug. 2024 – May 2025

- Wrote the simulation backend computing valid actions and state transitions for an autonomous LLM agent, plus a PyGame interface compiling object placement into thousand-line JSON task specs across 250+ scene configurations
- Implemented 25+ sync and async planning tasks to test agent reasoning over parallel dependencies and time delays

PROJECTS

Physically-Based Ray Tracer [GitHub] | C++, DirectX, HLSL, DXR, Intel Open Image Denoise — Jan. 2026 – Present

- Build a GPU-accelerated DXR path tracer with 32-bounce GI, Chiang fiber BCSDF, Disney BSDF, random-walk subsurface scattering, null-collision volumetrics, HDR environment lighting, and balance-heuristic MIS
- Design a DXR AnyHit-only shadow path with payload Fresnel for transparent and dielectric occluders
- Accelerate volume rendering with majorant-mip empty-space skipping and analytic fast-path on constant extinction
- Profile GPU bottlenecks with Nsight Graphics and a custom GPU-timestamp harness; shrink the ray payload by 65%

Eta Language Compiler in C++ [GitHub] | C++, Bison, x86, Docker — Jan. 2026 – May 2026

- Built an end-to-end compiler within a containerized Docker environment for a C-like statically-typed language, spanning lexical analysis, parsing, type checking, IR lowering, x86-64 code generation, and optimizations with a team of 4
- Implemented an SSA-based optimization framework over the IR driving worklist copy/constant propagation, constant folding, and dead-code elimination, reducing benchmark runtime by 60% and emitted instruction count by 35%
- Built a maximal-munch x86-64 instruction selector using a composable template-based pattern-matching DSL that exploits hardware addressing modes, plus trace scheduling that reorders basic blocks to maximize fall-through

CUDA Graphics Rasterizer Engine [GitHub] | C++, CUDA, SDL2 — Jul. 2025 – Dec. 2025

- Built a software rasterizer in C++ implementing the full graphics pipeline with no external graphics libraries
- Accelerated pipeline throughput by ∼100x over CPU baselines with 500k+ triangles using CUDA, optimizing workload orchestration by batching operations into single-dispatch kernels to eliminate launch overhead.
- Benchmarked via Nsight Compute, coalescing framebuffer writes through tile binning to improve global store throughput

Computer Science Content Creator [Instagram] — Aug. 2019 – Present

- Built an audience of 20K+ followers and 3M+ video views by posting computing topics, personal projects and digital art

SKILLS

Programming Languages: C++, Python, Rust, Java, C#, Javascript/Typescript, SQL
GPU Computing & Graphics: CUDA, DirectX, HLSL, WGPU/WebGPU, OpenGL/WebGL, Nsight Graphics, Nsight Compute
Development Tools: Linux, Git, Kubernetes, Docker